Homework Assignment No. 2

Neural Sentiment Analysis and Masked Language Model Pretraining

Released: 10/16/2026 Total: 100 points

Due: 11/10/2026

This assignment continues the product-review sentiment task from HW1. Compare a mean-pooled feedforward neural network (FFNN), a long short-term memory (LSTM) classifier, and a small Transformer encoder. Then pretrain the encoder with masked language modeling (MLM) and fine-tune it for sentiment classification.

Submission. Submit one Python file, `hw2.py`, to the HW2 assignment on Brightspace. Use the supplied starter and complete its **14 TODO blocks**. You may add helper functions, but do not change the provided constructors, function signatures, or experiment driver. Do not edit or submit `hw2_support.py`. No written report, notebook PDF, dataset, or model checkpoint is required.

Run `python hw2.py` with `data.tsv` and the unchanged `hw2_support.py` in the current folder. Use Python 3.12 and the versions in `requirements.txt`; install them before running. Your submission must not install packages or download data or models during execution. Importing `hw2.py` must not read data, train models, or print output. The required experiment runs on CPU; a GPU is not required.

Complete the assignment individually and follow course policies. All graded items are checked through function results and script outputs. Hidden tests may use smaller valid inputs. Return computed values, not hard-coded results. There is no minimum accuracy, F1-score, or loss target, and no required ordering of model scores. Required outputs are on page 7; function interfaces are on page 8.

Provided Data Preparation

Use the same original Amazon US Office Products reviews file as HW1, named `data.tsv`. Do not reuse HW1's processed features or `sentiment.py`. If the original data file is unavailable, contact course staff; do not substitute another category or version.

The supplied helper applies HW1's TSV parsing and valid-review rules. It maps ratings 4–5 to positive (1) and ratings 1–2 to negative (0), and discards rating 3. It samples 5,000 reviews per class without replacement using `random_state=42` separately for each class, concatenates positive then negative reviews, and resets the index. It takes a stratified 10% test split, followed by a stratified 1/9 validation split from the remaining rows, both with `random_state=42`. This gives **8,000 training, 1,000 validation, and 1,000 test reviews**: a new, smaller split, not HW1's exact split.

The supplied tokenizer decodes HTML entities, lowercases text, removes tags and URLs, normalizes curly apostrophes, and extracts tokens with the expression below. It retains stop words and contractions and does not lemmatize. Preserve word order and reviews with empty token lists.

```
r"[a-z]+(?:'[a-z]+)?"
"These pens aren't reliable." -> ["these", "pens", "aren't", "reliable"]
```

The driver builds the vocabulary from only the first 64 tokens of each training review; all models use at most the first 64 tokens of each review. `toy_data.tsv` is synthetic and is only for local testing. It must not replace the full-data experiment. No pretrained embeddings or external model downloads are used.

1. Vocabulary and Mini-batches (10 points)

(a) **Training vocabulary (4 points).** Implement `build_vocab`. Count exactly the supplied training tokens and retain words occurring at least `min_freq=2` times. Sort eligible words by decreasing frequency, breaking ties alphabetically, and keep at most `max_words=5000` ordinary words. Reserve `<pad>=0`, `<unk>=1`, and `<mask>=2`; assign ordinary word IDs 3, 4, ... in the sorted order. The cap excludes the three special tokens. Do not use validation or test text or any labels.

(b) **Encoding (3 points).** Implement `encode_reviews`. Keep each review's first `max_length=64` tokens and map words missing from the vocabulary to `<unk>`. Represent an empty review by `[1]`. Return unpadded lists of integer IDs in the original review order. Do not drop reviews, extend the vocabulary, or modify the inputs.

(c) **Mini-batches (3 points).** Implement `collate_batch`. Given a nonempty list of `(encoded_sequence, label)` pairs, right-pad each sequence with ID 0 to the longest sequence in that batch. Return `(ids, mask, lengths, labels)`. Use `torch.long` for IDs, lengths, and labels, and `torch.bool` for the mask. `True` marks a real (nonpadding) position. Lengths count tokens before padding; every input sequence contains at least one real token.

Example. For `([3, 4], 1)` and `([1], 0)`, return:

```
ids     = [[3, 4], [1, 0]]                # shape [B, T]
mask    = [[True, True], [True, False]]   # shape [B, T]
lengths = [2, 1]                          # shape [B]
labels  = [1, 0]                          # shape [B]
```

Notation. `B` is the batch size; `T` is the padded length of the current batch; `d` is a hidden-vector width; and `V` is the vocabulary size, including the three special tokens. Indices are zero-based. No review is entirely padding. Model dimensions shown below are the default configuration; use the supplied tensor and layer dimensions rather than hard-coding them.

2. Mean-Pooled Feedforward Classifier (10 points)

(a) **Masked mean (5 points).** Implement `masked_mean`. For each review, average the hidden vectors at real positions only. The inputs have shapes `[B, T, d]` and `[B, T]`; return `[B, d]`. Exclude padded values from the sum and padded positions from the denominator, even if their hidden vectors are nonzero.

```
pooled[b] = sum_t(mask[b,t] * hidden[b,t]) / sum_t(mask[b,t])
```

(b) **Forward pass (5 points).** Implement `MeanFFNN.forward` using its provided layers in this order:

```
X = embedding(ids)                # [B, T, 32]
r = masked_mean(X, mask)           # [B, 32]
h = ReLU(hidden(r))                # Linear(32, 64)
logits = output(h)                 # Linear(64, 2)
```

Return raw logits of shape `[B, 2]`, not probabilities or class labels. Columns 0 and 1 correspond to labels 0 and 1. Embeddings are trained jointly with the classifier; the padding embedding starts at zero and remains zero. Do not add positional embeddings, extra layers, dropout, or pretrained vectors.

Implementation note. In PyTorch, an `nn.Module` stores trainable layers, and its `forward` method defines the computation. A logit is an unnormalized class score. `nn.Linear` applies the same transformation to the last tensor dimension. `mask[..., None]` changes a mask of shape `[B, T]` to `[B, T, 1]`, allowing it to apply to every coordinate of a token vector.

Local check. Vectors `[2, 4]`, `[4, 8]`, and a padded `[100, 100]` have masked mean `[3, 6]`. Reordering the real tokens does not change the mean-pooled representation.

3. LSTM Sequence Classifier (15 points)

Implement `LSTMClassifier.forward` using the provided trainable embeddings, one unidirectional `nn.LSTM` with `batch_first=True`, and the output layer. The default embedding and LSTM hidden widths are both 32. You do not need to implement the LSTM gates or backpropagation from scratch.

(a) **Sequence computation (6 points).** Pass the embeddings through the LSTM. Embedding inputs have shape `[B, T, d_model]`; the sequence of hidden states has shape `[B, T, hidden_dim]`. Use zero initial hidden and cell states on every call, as `nn.LSTM` does when those states are omitted. Do not carry state between batches.

(b) **Last real hidden state (6 points).** For review `b`, select the hidden state at index `lengths[b] - 1`. Do not select the final padded position. Because the LSTM is unidirectional, later padding does not affect an earlier real-token state. Running the padded batch and selecting each last real hidden state is sufficient; packed-sequence utilities are not required.

(c) **Classification (3 points).** Apply the provided output layer to the selected states and return raw logits of shape `[B, 2]`. Do not average the states, use the cell state as the review representation, or apply sigmoid or softmax.

Local check. For lengths `[4, 2]`, select `states[0, 3, :]` and `states[1, 1, :]`.

4. Training and Evaluation (15 points)

Implement the following functions for all three classifier classes. Use the supplied model, data loader, and optimizer rather than constructing replacements inside these functions.

(a) **One training epoch (8 points).** Implement `train_one_epoch`. Call `model.train()`. For each batch, clear old gradients, compute logits and mean cross-entropy loss against the integer labels, call `loss.backward()`, and then call `optimizer.step()`. Process every example exactly once, including the final smaller batch. Return a Python float: the sum of each batch's pre-update mean loss multiplied by its batch size, divided by the total number of examples. Do not average batch losses without weighting by batch size.

(b) **Evaluation (7 points).** Implement `evaluate_model`. Save the model's current training/evaluation mode, call `model.eval()`, and use `torch.no_grad()`. Make no optimizer updates. Aggregate cross-entropy by total example count and compute metrics from predictions over the entire split, not by averaging batch metrics. Predict using `argmax` over the two logits; a tie selects class 0. Return unrounded Python floats under the keys `loss`, `accuracy`, `precision`, `recall`, and `f1`. Use binary averaging, positive label 1, and `zero_division=0` for precision, recall, and F1-score. Restore the original training/evaluation mode before returning. Do not change model parameters, stored gradients, input data, or data order.

Cross-entropy and gradients. `torch.nn.functional.cross_entropy` takes raw logits of shape `[B, 2]` and integer labels of shape `[B]`; it applies log-softmax internally. `loss.backward()` computes gradients automatically. Gradients accumulate unless cleared, so call `optimizer.zero_grad()` before each backward pass. Training/evaluation mode and gradient recording are separate controls.

Provided training procedure. The helper supplies the Adam optimizer; its derivation is not required. `fit_classifier` runs five supervised epochs and restores the checkpoint with the lowest validation loss. A checkpoint is a saved copy of model parameters. Exact ties retain the earlier epoch. Checkpoint management is provided, not a TODO. Validation data is used only for checkpoint selection; test data must not affect vocabulary construction, training, pretraining, or checkpoint selection.

5. Self-Attention and Transformer Encoder (25 points)

Use one attention head and one encoder block. Implement the specified computations with the provided layers; do not replace them with a different Transformer architecture.

(a) **Scaled dot-product attention (10 points).** Implement `scaled_attention`. Inputs `q` and `k` have shape `[B, T, d]`; `v` has shape `[B, T, dv]`. Return `(context, weights)` with shapes `[B, T, dv]` and `[B, T, T]`. Use matrix multiplication, masking, and softmax directly. Do not use `nn.MultiheadAttention`, a built-in Transformer layer, or a fused attention function.

```
scores = (q @ transpose_last_two_dims(k)) / sqrt(d)
# Apply the padding and optional causal masks to scores.
weights = softmax(scores, dim=-1)
# Set the weights of padded query rows to zero.
context = weights @ v
```

Before softmax, replace scores for padded keys with negative infinity. When `causal=True`, also block key index `j` whenever `j > i`, where `i` is the query index. Apply softmax over keys (the last dimension). Then set every padded query row of the weights to zero before computing the context. Real query rows sum to one; padded query rows and their context vectors are zero.

Masks have a nonempty real-token prefix followed by right padding. Thus every query row has at least one allowed key before padded query rows are zeroed, including in causal mode. Use `causal=False` for sentiment classification and MLM; the causal option is tested separately. The padding mask is required in both modes.

(b) **Encoder block (12 points).** Implement `MiniEncoder.forward` in the following order. Each name refers to the corresponding provided layer. For example, `q_proj(X)` means calling that `nn.Linear` layer, not manually transposing its stored weight.

```
X = embedding(ids) + position(arange(T))
X = X * mask[..., None]
Q, K, Z = q_proj(X), k_proj(X), v_proj(X)
C, A = scaled_attention(Q, K, Z, mask, causal=False)
H1 = norm1(X + out_proj(C))
H2 = norm2(H1 + ff2(ReLU(ff1(H1))))
H = H2 * mask[..., None]
```

Return `H` of shape `[B, T, d_model]`, with default width 32. The learned position table has 64 entries indexed from 0, and position vectors are broadcast across the batch. Attention projections are 32-to-32 linear layers without bias. The position-wise feedforward layers are 32-to-64-to-32 with biases. Apply each residual addition before its `LayerNorm(32, eps=1e-5)`. Do not change this order, omit a residual connection, add blocks, or add dropout.

Layer normalization. Use the supplied `nn.LayerNorm` modules. Each normalizes a token vector across its hidden coordinates, not across tokens or reviews. The small epsilon prevents division by zero. You do not need to implement layer normalization from scratch.

(c) **Sentiment classification (3 points).** Implement `TransformerClassifier.forward`. Apply its encoder, take the masked mean of the output token vectors, and apply the provided `Linear(32, 2)` output layer. Return raw logits of shape `[B, 2]`.

Local check. With zero query and key vectors, values `[2, 0]`, `[0, 4]`, and padded `[90, 90]` give noncausal context `[1, 2]` at each real position. Causal attention gives `[2, 0]` at the first position and `[1, 2]` at the second. The padded query has zero context.

6. MLM Pretraining and Fine-tuning (20 points)

Masked language modeling trains an encoder to predict selected original words from a corrupted input. Pretrain using only the same training reviews, ignoring their sentiment labels. Then fine-tune the encoder and a binary sentiment head jointly. Do not use validation or test reviews for pretraining, even with their labels removed.

This is a small-scale exercise, not a full BERT implementation. Use word tokens, one attention head, one encoder block, and MLM only. Do not add WordPiece tokenization, a `[CLS]` token, next-sentence prediction, or an external pretraining corpus.

(a) **Input corruption and targets (10 points).** Implement `corrupt_for_mlm`. The caller supplies `ids` and three tensors of the same shape: `select_u` and `replace_u` contain values in `[0, 1)`, and `random_ids` contains ordinary replacement word IDs. Do not generate random numbers inside this function.

A position is eligible only if its original token ID is at least 3. Select an eligible position when `select_u < mask_prob`, with default `mask_prob=0.15`. Set its target to the original ID; set every other target to -100, meaning "ignore this position in the loss." Never select `<pad>`, `<unk>`, or `<mask>`. Return a corrupted copy of the input IDs and a separate target tensor without modifying any supplied tensor.

For each selected position, apply the following rule. Leave unselected input positions unchanged:

```
replace_u < 0.8          -> replace with <mask> (ID 2)
0.8 <= replace_u < 0.9   -> use the supplied random_ids value
0.9 <= replace_u < 1.0   -> keep the original token ID
```

Every selected position contributes to the loss, including an unchanged token or a random replacement equal to the original token. Selection is independent per eligible token: do not force exactly 15% of tokens to be selected or require a target in every review or batch. Keep the original padding mask; a corrupted `<mask>` token is still a real position.

(b) **Selected-token loss (5 points).** Implement `masked_token_loss`. The last logit dimension has size `V`; the target tensor has the matching leading dimensions. Compute mean cross-entropy over targets not equal to -100. Use the full `V`-class softmax, including special-token columns. Do not divide by the number of padded or unselected positions. If all targets are ignored, return a differentiable scalar zero, such as `logits.sum() * 0.0`, rather than NaN.

(c) **Encoder transfer (5 points).** Implement `transfer_encoder`. Return a deep, independent copy of the template classifier. Load the pretrained encoder's weights into that copy, leaving its sentiment-head weights unchanged. Make all returned parameters trainable. Do not modify or share parameter storage with either input. Fine-tuning updates both the encoder and the sentiment head; it does not freeze the encoder.

Provided pretraining procedure. The helper runs two MLM epochs with a separate `Linear(32, V)` vocabulary-prediction head. For efficiency, it applies this head only to selected encoder outputs; this gives the same loss as ignoring unselected logits. It skips batches with no selected targets and reports zero if an entire epoch has no targets. Discard the MLM head after pretraining. The sentiment head has two outputs, not `V`.

Masking terminology. A padding mask excludes nonexistent positions. A causal mask excludes future context. MLM corruption changes selected input tokens and creates reconstruction targets. These are different operations; MLM uses a padding mask but no causal mask.

7. Experiment Integration (5 points)

The supplied `run_experiment` defines the experiment and prints the required results. **Do not modify it.** Complete the TODOs so the driver runs with both the full-data defaults and smaller valid test inputs. No additional training pipeline, parameter search, written analysis, or model architecture is required.

(a) **Function interfaces (2 points).** Keep all required signatures. Use the input tensor shapes and constructor dimensions so functions work on smaller valid inputs. Functions must be independently callable and compute their results without saved outputs or a prior experiment run. Preserve caller-owned data except for the model and optimizer updates explicitly required in Section 4(a). Importing the module must have no data-loading, training, or printing side effects.

(b) **Full pipeline and outputs (3 points).** The unchanged driver must print the 33 computed lines on page 7. Do not add debug output, extra epochs, runtime downloads, automatic sample-size reductions, or a substitute dataset. The synthetic smoke test is a separate local check, not the required full-data run.

Fixed Experiment Settings

Data and vocabulary. Use 5,000 reviews per binary class, split into 8,000 training, 1,000 validation, and 1,000 test reviews. Use the first 64 tokens per review. Build the vocabulary from training tokens only, with minimum frequency 2 and at most 5,000 ordinary words. Include the three special tokens, so `V <= 5003`.

Models and mini-batches. Use batches of 64 reviews, right-pad within each batch, and keep the final smaller batch. Embedding and encoder widths are 32. The FFNN hidden width is 64, the LSTM hidden width is 32, and the encoder feedforward hidden width is 64. Use the supplied one-head, one-block Transformer, with no dropout, learning-rate scheduler, or gradient clipping.

Training. Run five supervised epochs for each of `FFNN`, `LSTM`, `Transformer`, and `MLM-Transformer`. Select each model's checkpoint by validation loss. MLM pretraining runs for two epochs, using 15% token selection followed by the specified 80% / 10% / 10% replacement rules. The helper supplies Adam with learning rate 0.001, `betas=(0.9, 0.999)`, `eps=1e-8`, and no weight decay.

Execution and seeds. Run on CPU with float32 model parameters and two PyTorch CPU threads. Keep all supplied seeds. Each supervised run receives a fresh training-data loader seeded 42, producing the same epoch-by-epoch review shuffling order. The driver resets initialization seeds and supplies a separate seed for MLM pretraining.

Transformer Comparison

The driver creates one untouched Transformer template. The Transformer trained from scratch and the encoder used for MLM pretraining start with identical encoder parameters. After pretraining, the fine-tuned classifier uses the pretrained encoder and the same initial sentiment-head weights as the scratch model. Fine-tuning receives a fresh supervised optimizer; pretraining optimizer state is not transferred. Do not pretrain an encoder that has already received supervised updates.

All four configurations use the same data splits, vocabulary, sequence limit, batch size, and supervised epoch count. MLM adds two pretraining passes, so the two Transformer configurations do not use equal total training work. An improvement from pretraining is not required, and a single run on this small corpus does not establish a general advantage.

Final MLM loss. The reported value is the selected-token-weighted mean of pre-update batch losses from the final pretraining epoch. Weight each batch by its number of selected targets, not its number of reviews. The helper computes and preserves this value before fine-tuning. It is a training loss, not a test loss or autoregressive perplexity.

Required Script Outputs

Print exactly **33 lines** to standard output, in the order below. Use the labels exactly as written. Replace each angle-bracket placeholder with its computed value. Counts and epoch numbers are integers. Losses and metrics use four decimal places; metrics are on a 0–1 scale. Do not print sample reviews, explanations, setup messages, extra blank lines, or debug output.

1. Data (4 lines). The default split sizes are 8000, 1000, and 1000. For smaller valid test inputs, print the actual sizes. Vocabulary size includes all three special tokens.

```
Training reviews: <integer>
Validation reviews: <integer>
Testing reviews: <integer>
Vocabulary size: <integer>
```

2. Classifier results (28 lines). Print the following seven-line block four times, replacing `<Model>` with, in order: **FFNN**, **LSTM**, **Transformer**, and **MLM-Transformer**. Epochs are numbered from 1. Validation Loss is the loss of the selected checkpoint; compute test results only after restoring that checkpoint.

```
<Model> Best epoch: <integer>
<Model> Validation Loss: <value>
<Model> Testing Loss: <value>
<Model> Testing Accuracy: <value>
<Model> Testing Precision: <value>
<Model> Testing Recall: <value>
<Model> Testing F1-score: <value>
```

3. MLM pretraining loss (1 line). Print the selected-token-weighted training loss from the final MLM epoch, as defined in Section 7. This is not a held-out evaluation loss.

```
MLM final training loss: <value>
```

Before Submitting

After completing the TODOs, run these commands from the assignment folder:

```
python -m unittest -v test_public
python run_smoke.py
python hw2.py
```

The first two commands are local checks; they do not replace the full-data run. Passing public tests does not guarantee full credit. Run the final command in a fresh process with the original `data.tsv`, and confirm that it prints exactly 33 lines. Confirm that `import hw2` is silent, does not read data or train models, and that each required function can be called independently. Do not use absolute file paths or runtime downloads. Submit **only** `hw2.py`; the grader provides the unchanged support module and environment.

Section	1	2	3	4	5	6	7	**Total**
Points	10	10	15	15	25	20	5	**100**

Required Function Interface

Implement the following functions and methods in `hw2.py`. Only the provided `run_experiment` prints output. Return full-precision values; round only printed results. Do not modify caller-owned inputs except where training explicitly requires updates. Hidden tests obey the stated input conditions; arbitrary malformed-input handling is not required. All tensors are on CPU. Floating-point inputs are finite; numerical tests may use float64. Preserve gradients through differentiable computations.

`build_vocab(train_tokens, max_words=5000, min_freq=2)`
Return `dict[str, int]`. Input is a nonempty list of token lists, which may individually be empty. Ordinary tokens never equal the three special-token strings. Both parameters are positive integers. Count exactly the supplied tokens; the driver truncates reviews before this call. Always include the three special IDs, even if no ordinary word qualifies.

`encode_reviews(token_lists, vocab, max_length=64)`
Return unpadded integer-ID lists in input order. Input contains at least one review; an individual review may be empty. `max_length` is positive. Use the supplied vocabulary and represent an empty result as `[1]`.

`collate_batch(examples)`
Return `(ids, mask, lengths, labels)` with shapes `[B, T]`, `[B, T]`, `[B]`, and `[B]`. Each example is a nonempty list of positive IDs paired with a binary integer label. Uncorrupted sequences never contain ID 0 or 2. The Boolean mask equals `ids != 0`; all other returned tensors use `torch.long`.

`masked_mean(hidden, mask)`
Inputs have shapes `[B, T, d]` and `[B, T]`. Return `[B, d]`. Each mask row has at least one `True` value. Explicitly ignore padded hidden values, which need not be zero.

`scaled_attention(q, k, v, mask, causal=False)`
Inputs have shapes `[B, T, d]`, `[B, T, d]`, `[B, T, dv]`, and `[B, T]`. The Boolean mask has a nonempty real-token prefix in each row. Return `(context, weights)` with shapes `[B, T, dv]` and `[B, T, T]`. Return weights after padded query rows are zeroed, not attention scores.

`MeanFFNN(vocab_size, d_model=32, hidden_dim=64)`
`LSTMClassifier(vocab_size, d_model=32, hidden_dim=32)`
`MiniEncoder(vocab_size, d_model=32, ff_dim=64, max_length=64)`
`TransformerClassifier(encoder)`
Constructors are provided; implement only the four `forward` methods. Classifiers use `forward(ids, mask, lengths)` and return `[B, 2]` raw logits. `MiniEncoder.forward(ids, mask)` returns `[B, T, d_model]`. Inputs follow the batch conventions; an encoder may also receive MLM-corrupted IDs. `T` never exceeds the position-table length. Tests may use smaller layer dimensions and batch sizes; do not hard-code 32 or 64.

`train_one_epoch(model, loader, optimizer)`
`evaluate_model(model, loader)`
Each loader is a finite, reusable iterable containing at least one collated batch. Models use the classifier forward signature. Training updates the supplied model and optimizer and returns a Python float. Evaluation returns Python floats under `loss`, `accuracy`, `precision`, `recall`, and `f1`, and restores the prior training/evaluation mode. Evaluation data may contain only one label class.

`corrupt_for_mlm(ids, select_u, replace_u, random_ids, mask_prob=0.15)`
All input tensors have shape `[B, T]`. `ids` and `random_ids` use `torch.long`; selection and replacement values are floating point in `[0, 1)`. Require `0 < mask_prob <= 1`. Ordinary replacement IDs lie in `[3, V)`; if no ordinary words exist, their unused entries may be zero. Return two independent `torch.long` tensors of the same shape: corrupted IDs and targets.

`masked_token_loss(logits, targets)`
Return a scalar tensor. Logits have shape `[B, T, V]` or `[K, V]`, where `K` is the number of supplied token positions. Targets have the matching leading shape and contain -100 or valid vocabulary IDs. Leading dimensions are nonempty; the all-ignored case is valid.

`transfer_encoder(pretrained_encoder, template_classifier)`
The encoder architectures match. Return a new `TransformerClassifier` with the pretrained encoder weights and unchanged template sentiment-head weights. Preserve both inputs and make the returned parameters trainable.